

legacy-reverse マニュアル

目次

2.1 skill 側 — 全プロジェクト共通（固定）	3
2.2 プロジェクト側 — ここが可変	3
4.1 4 つの置き場所の使い分け	6
4.2 conventions.md — 規約	6
4.3 domain-knowledge.md — 業務知識	6
4.4 docs/templates/ — 仕様書の項目立て	7
4.5 docs/prompts/ — 工程別の AI への指示	7
4.6 共通のルール	7
無人バッチ（pipeline.py）の挙動	9

レガシーコード（Fortran / C / C++ ほか）を Python へ仕様ベースで移植する Claude Code skill 群です。レガシーを読んで直接書き写すのではなく、間に仕様書を挟みます。

①解析 → ②変数辞書 → ③仕様書 → ④テスト仕様 → ⑤テストコード → ⑥実装 → ⑦テスト → ⑧完了検証 → ⑨分析・改善

土台の決めごとは3つ。全プロジェクト共通で、プロジェクト側からは変えられません（変えるには skill 自体の改版が必要です）。

Table 1

決めごと	中身
クリーンルーム分業	②③はレガシーを見ない。③は②だけ、④は①だけを入力に作り、⑤で突き合わせる。仕様書の穴はテスト失敗として必ず表面化する
ハッシュ連鎖	①→②→③が上流のハッシュを持つ。上流を直すと下流が自動で「要再生成 (stale△)」になる
人の承認ゲート	仕様の確定・テスト仕様の承認・⑤の裁定は人。AI は「仮説+Yes/No の問い」で聞き、勝手に確定しない

人の仕事はトリガを引く / 質問に答える / 承認する / 裁定する の4つです。そのうえで「このプロジェクトではどう書くか」を人が MD に書いて調整します（§4）。本書の中心はそこ、そのファイルが誰にいつ読まれるか（§3）です。

Table 2

読みたいこと	文書
初めて使う（操作を順番に）	slides/index.html
コマンド即引き	QUICKREF.md
構成・参照関係・人が書く MD	本書
層の分け方とスクリプトの責務	ARCHITECTURE.md
そう設計した理由	DESIGN.md
skill が従う規則・データの正	references/workflow.md / references/schema.md

以降 <LR> = skill の置き場所（<project>/[.claude/skills/legacy-reverse](#)）、`ledger = python <LR>/scripts/ledger.py` の略記です。

2. ディレクトリ構成

2.1 skill 側 — 全プロジェクト共通（固定）

```
legacy-reverse/
  SKILL.md                # 全体管理（状況表示・次の一手・セットアップ）
  skills/
    legacy-0-analyze/     ① 解析（関数リスト・規約のヒアリング）
    legacy-0-dict/        ① 変数辞書（1 変数=1 語義を人が承認）
    legacy-1-spec/        ① 仕様書（レガシーを読める唯一の工程）
    legacy-2-testspec/    ② テスト仕様（①だけを入力）
    legacy-3-testcode/    ③ テストコード（②だけを入力）
    legacy-4-impl/        ④ 実装（①だけを入力）
    legacy-5-test/        ⑤ テスト実行・トリアージ
    legacy-6-check/       ⑥ 完了検証
    legacy-7-analyze/     ⑦ 分析・改善（挙動保存）
  references/             # 全 skill が従う共通規則（skill が読む）
    workflow.md           情報遮断・ハッシュ連鎖・承認・ISSUE・固変分離の規則＝規則の正
    schema.md             プロジェクト構成とデータスキーマ＝データの正
    graph-dict-design.md   グラフ層・変数辞書・例外ポリシーの設計
  scripts/               # 決定的な処理（LLM を使わない。単体でも動く）
  hooks/                 # 物理的な禁止（④⑤中の tests/ 編集拒否など）
  assets/templates/      # プロジェクトへ配るシード（下の docs/templates・docs/prompts の元）
  ARCHITECTURE.md / DESIGN.md / MANUAL.md / QUICKREF.md / slides/
```

スクリプト 1 本ずつの責務は [ARCHITECTURE.md §2](#) にあります。本書では触れません。

2.2 プロジェクト側 — ここが可変

【人】 = 人だけが書く／【AI】 = AI が書き人が承認／【機械】 = 自動生成（手編集禁止）。

```
<project>/
  legacy/                レガシー原文（読めるのは①と②だけ）
  src/ tests/            【AI】 ④実装 / ③テストコード
  data/                 【機械】 functions.json・ledger.json・variables.json ほか
  docs/
    templates/          【人】 仕様書の"項目立て" (spec.md / test-spec.md) ← §4.4
    prompts/            【人】 工程別の"AI への指示" (1-spec / 2-testspec /
                        3-testcode / 4-impl .md) ← §4.5
    conventions.md       【人】 規約（型対応・丸め・命名・docstring...） ← §4.2
    domain-knowledge.md  【人】 業務知識・語彙・ISSUE 回答の蓄積 ← §4.3
    exception-policy.md  【人】 例外の決定（EP-xxx。add-policy コマンドで追記）
    review-feedback.md   【人】 修正依頼（AI が次回起動時に拾う）
    issues/ISSUE-xxx.md  【AI】 本文。「回答（人が記入）」欄だけ【人】
    specs/ test-specs/   【AI】 ①仕様書 / ②テスト仕様
    test-results/        【AI】 ⑤結果報告書
    index.qmd wbs/       【機械】 WBS（進捗のホーム）
    spec-review.md       【機械】 ①の一斉レビュー表
    variables.qmd        【機械】 変数辞書ページ
    exception-queue.md    【機械】 未決定 hazard の質問キュー
    completion-check.md  【機械】 ⑥完了検証レポート
    _site/              【機械】 HTML 出力
```

【人】のファイルに AI は書き込みません（提案文の提示までです）。あなたの意思の一次記録を AI の文章と混ぜないため、ここが混ざると「誰が決めたのか」が後から追えなくなります。

3. 参照関係 — 誰が何を読むか

工程を起動すると、skill は毎回この順で読みます。

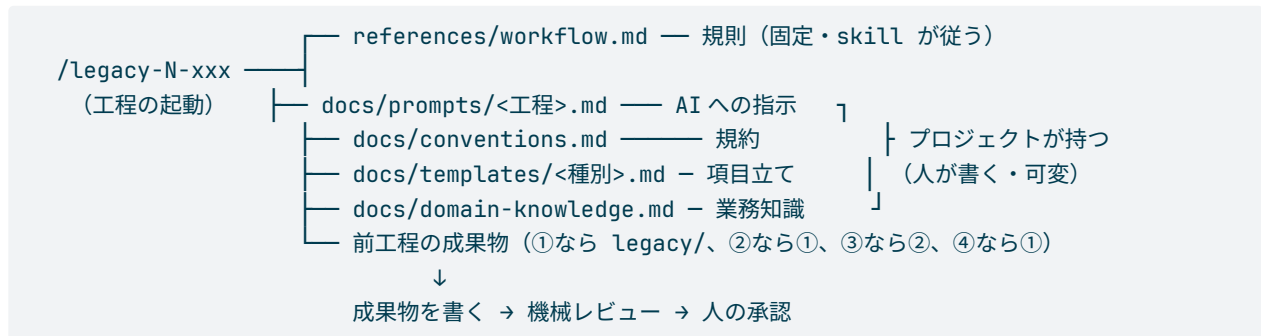


Table 3

工程	読む（前工程の成果物）	読む（人が書いた可変ファイル）	出力
① 解析	legacy/ 全部	—（ここで conventions.md と語彙を人が書く）	functions.json・骨子・WBS
② 辞書	機械が集めた根拠のみ	domain-knowledge.md	variables.json
① 仕様書	legacy/ 該当関数・functions.json	prompts/1-spec.md・conventions.md・templates/spec.md・domain-knowledge.md	docs/specs/
② テスト仕様	①(reviewed)	prompts/2-testspec.md・conventions.md・templates/test-spec.md・domain-knowledge.md	docs/test-specs/
③ テストコード	②(approved)	prompts/3-testcode.md・conventions.md	tests/
④ 実装	①(reviewed)	prompts/4-impl.md・conventions.md	src/
⑤ テスト	実行結果・①②	—	docs/test-results/

読むときの規則は 3 つです。

- **情報遮断:** 表に無いものは読みません（②③④はレガシー原文を見ない、③は①を見ない、④は②と tests/ を見ない）。触れたくなったらそれは仕様の穴なので、ISSUE を立てて止まります
- **毎回読み直す:** 作り直し・改訂・バッチ実行・無人実行でも同じです。「骨子を作ったときの内容」で固定されないの、**あとから可変ファイルを直すと、次に動かした関数から効きます**
- **優先順位は 固定契約 > skill の手順 > プロジェクト個別指示。** 可変ファイルに土台（§1 の 3 つ・必須項目・禁止事項）を覆す指示があっても、AI は従わずにあなたへ報告します

4. 人が書くファイル

4.1 4 つの置き場所の使い分け

同じ「AI にこう書いてほしい」でも、**射程で置き場所が違います**。ここを間違えると効きません。

Table 4

書きたいこと	置き場所	効く範囲
節を足したい・消したい（仕様書の構成）	<code>docs/templates/spec.md</code> <code>test-spec.md</code>	①②
型の対応・丸め・命名・docstring などの規約	<code>docs/conventions.md</code>	①②③④
重点・書き方の癖・繰り返したくない指摘	<code>docs/prompts/<工程>.md</code>	その工程
業務知識・略語・過去の判断	<code>docs/domain-knowledge.md</code>	①②③
0 割など例外の扱いの決定	<code>docs/exception-policy.md</code> (<code>hazards.py</code> <code>add-policy</code>)	①②（機械レビューが突合）
この関数のこの箇所だけ直して	チャット or <code>docs/review-feedback.md</code>	その 1 件

これらは①の最初に `ledger init-templates` が雛形をまとめて作ります（既存は上書きしません）。どのファイルにも空欄と「何を書くか」の記入ガイドがコメントで入っているので、白紙から書き始める必要はありません。書けているかは `ledger authored` で一覧できます。

× 未作成 まだファイルが無い（`init-templates` で作る）
・ 未記入 雛形のまま（異常ではない。`prompts` なら「個別指示なし」の意味）
▲ 記入途中 `{{...}}` のプレースホルダが残っている
✏ 記入あり 人が書いた

4.2 conventions.md — 規約

①で AI が質問リストを出すので、**あなたが記入して確定**します。節は次のとおりです。

型対応表 / 数値の丸め・比較規則 / 単位・スケール / 日付・時刻 / 文字コード・外部ファイル / 既知レガシーバグの扱い / ディレクトリ・命名 / モックの書き方 / テストケース ID の対応付け / docstring 規約 / 禁止事項

- ・ 後戻りが高くつくのは「丸め・許容誤差・単位・日付・文字コード・既知バグの扱い」です。②以降で全ケースの期待値の前提になるため、覆ると作り直しになります。①で決め切ってください
- ・ 迷ったら「関数ごとに違う」ではなく**全体の既定を 1 つ**決めます。例外は `domain-knowledge.md` に関数単位で書けば、そちらが勝ちます

4.3 domain-knowledge.md — 業務知識

- ・ ①で「語彙・略語集」を先に埋めると、変数辞書の精度が上がります（略語が既知語と一致した変数は一括承認候補に回り、あなたが 1 件ずつ見る量が減ります）
- ・ ISSUE に答えた内容は、AI が転記文を提案するので**あなたが貼って確定**します。蓄積すると同じ質問が来なくなります

4.4 docs/templates/ — 仕様書の項目立て

spec.md (①) と test-spec.md (②) は節の構成と、各節の記入ガイドです。骨子生成でこの本文が写され、機械レビューは「テンプレにある # 見出し」を必須節として検査します。節を足す・改名する・削除するのは自由ですが、次の固定契約だけは動かさせません。

Table 5

対象	契約
① 置換マーカー	LR:IO-TABLES • LR:CALLS-TABLE • LR:HAZARD-TABLE (機械が表を差し込む位置)
① 契約見出し	# 機能詳細 / # 副作用・例外 + ## 例外・数値特異点 / # 未確定事項
② 契約見出し	# トレーサビリティマトリクス

- 節の見出し行末に LR:OPTIONAL (HTML コメント形式) を付けると任意節になります
- 直したら `python <LR>/scripts/review_checks.py template --root .` で契約を確認できます (骨子生成も契約違反なら止まります)
- 記入ガイドは HTML コメントで書きます。AI は充填後にガイドを消します

4.5 docs/prompts/ — 工程別の AI への指示

①～④に 1 ファイルずつ、起動のたびに読まれる上乗せ指示です。用意されている節:

重点 / 用語・表記の約束 / 繰り返さないでほしい指摘 / 手本にする成果物

```
ledger init-templates      # 雛形を配置 (④の最初。既存は上書きしない)
ledger authored            # 人が書くファイルの記入状況を一覧
```

- 雛形のまま (案内コメントだけ) なら「個別指示なし」として扱われます。必要な工程だけ書けば十分です
- 書けるのは「どう書くか」だけです。工程の順序・読んでよい入力・必須項目・承認の要否は変えられません (各ファイルの冒頭に「ここに書いても効かないもの」を明記してあります)
- 手本にする成果物が一番効きます。「粒度はこの仕様書に揃えて」と 1 行書くほうが、抽象的な指示を 10 行書くより揃います

使い方のコツ: レビューで 2 回同じ指摘をしたら、その内容を「繰り返さないでほしい指摘」に 1 行足してください。3 回目からは指摘が要らなくなります。逆に、1 件だけの直しはチャットで伝えれば十分です (そのために書き足すと指示が太っていきます)。

4.6 共通のルール

- AI は【人】のファイルを書き換えません。直したほうがよいと判断したら提案文を出すので、採否はあなたが決めます
- 直した内容は次にその工程を動かした関数から効きます。既に完成した成果物は自動では書き換わりません (作り直すと反映されます)
- 何をどこに書いたかは、`ledger authored` と `git diff` で追えます。可変ファイルはすべてプロジェクトの git 管理下に置いてください

5. skill 側を触る人へ

「どのプロジェクトでも同じようにしたい」ものだけが skill 側の対象です。1 つのプロジェクトの都合は § 4 に書いてください。

Table 6

直したいこと	直す場所
工程の手順・禁止事項	skills/legacy-*/SKILL.md
全工程に効く規則（情報遮断・承認・ISSUE 運用）	references/workflow.md
データの形・ディレクトリ構成	references/schema.md
機械の判定（検査・台帳・抽出）	scripts/*.py
配布する雛形の初期内容	assets/templates/（prompts/ 含む）

守ってほしい約束が 2 つあります。

1. **skill に「規約の実体」を書かない。**「docstring は Google スタイル」のようなプロジェクトごとに変わる内容を skill に書くと、conventions.md を直しても効かなくなります。skill からは**節の名前で参照するだけ**にしてください
2. **直したら機械レビューを通す。**文書とスクリプトの食い違い（消したスクリプトへの参照、実在しないコマンド・オプション）を検出します

```
python <LR>/scripts/check_skill.py          # 指摘は file:line と直し方つき
python <LR>/scripts/check_skill.py --json    # AI に直させるとき
```

NG が出たら**ゼロになるまで直します**。原則は「スクリプトの argparse が正、文書を合わせる」。回歸テストは scripts/selftest/ にあります。

6. 工程ごとの要点

起動はすべて人です（AI が勝手に次へ進むことはありません）。

Table 7

工程	起動	あなたがやること	完了条件（機械が判定）
① 解析	<code>/legacy-0-analyze</code>	ヒアリングに答え、 <code>conventions.md</code> を確定	抽出の完全性突合
① 辞書	<code>/legacy-0-dict</code>	語義を承認（A/B は一括、C/D は直して承認）	全変数 approved
① 仕様書	<code>/legacy-1-spec F-xxxx</code>	内容をレビューして OK / 修正指示	status: reviewed
② テスト仕様	<code>/legacy-2-testspec F-xxxx</code>	△未確定に答えて承認	status: approved
③ テストコード	<code>/legacy-3-testcode F-xxxx</code>	（基本なし）	ケース ID とテストの突合
④ 実装	<code>/legacy-4-impl F-xxxx</code>	（基本なし）	スタブ検知ゼロ
⑤ テスト	<code>/legacy-5-test F-xxxx</code>	fail が 3 回続いたら裁定	実装率 100% ・ 失敗 0
⑥ 完了検証	<code>/legacy-6-check</code>	不足の埋め戻しを指示	全関数 pass
⑦ 分析・改善	<code>/legacy-7-analyze</code>	<code>bench.py</code> を用意し、施策を承認	テスト全 pass 維持

無人バッチ（`pipeline.py`）の挙動

まとめて流すときは `pipeline.py` を使います。サブコマンドは 4 つです。

Table 8

コマンド	対象	使いどころ
<code>pipeline.py spec</code>	①だけを全件	①を全部 draft にして、一斉レビュー表でまとめて承認する運転
<code>pipeline.py run</code>	①～⑤を工程横断	承認済みの関数から②③④⑤へ自動で進む。 <code>--only testspec</code> で工程を限定できる
<code>pipeline.py dict</code>	変数の解釈（①）	対象が関数ではなく 変数のチャンク （既定 40 件＝1 プロセス）
<code>pipeline.py priority</code>	★優先の設定・一覧	実行中でも 割り込み順を変えられる（次の 1 件から効く）

なぜ①だけ専用の `spec` があるのか（`run --only spec` でも①は回せます）：

- `spec` は書きかけ（機械レビュー NG の draft）を先に修復してから未着手へ進み、終了時に「レビュー待ち何件」を報告します。①→一斉レビューという運転に合わせた形です
- 対象の再走査が `spec` はチャンク境界ごと / `run` は 1 件ごとです。`run` は実行中の承認や ★ を即座に拾える代わりに毎回全関数を見るので、2000 関数級で①を全件流すときは `spec` が軽い

- 逆に言うと、規模が小さければ `run --only spec` で十分です。歴史的には `spec` が先にでき、あとから `run` が①～⑤へ一般化しました

共通の挙動（4 つとも同じ）：

- **1 関数 = 1 つの新しい headless プロセス** (`claude -p "/legacy-1-spec F-xxxx"`)。会話が積み上がらないので、何千関数でも 1 件あたりのトークンは一定です
- **完了判定は AI の申告ではなくファイルの状態** (status が draft になったか、機械レビューが NG ゼロか)。NG ならリトライし、駄目ならそのセッション中はスキップします
- **中断・再開は同じコマンド**。Ctrl-C でも電源断でも、進捗はファイル側にあるので続きから進みます
- **レート制限は失敗に数えず** 指数バックオフで待ちます（既定の待ち上限は合計 6 時間）。1 関数のタイムアウトは既定 30 分、**連続 3 件失敗したら環境異常とみなして停止**します
- 進捗は `/pipeline.html`（閲覧サイト）でライブに見えます。実行中でも人は並行して承認できます
- ログは `.legacy-reverse/agent-logs/<fid>.txt`（応答全文）と `.legacy-reverse/pipeline-log.jsonl`（1 行 1 試行）。失敗の一次情報はここです

```
python <LR>/scripts/pipeline.py run --root . --dry-run           # 対象と実行順の確認だけ
python <LR>/scripts/pipeline.py spec --root . --max-funcs 200    # ①を 200 件まで
python <LR>/scripts/pipeline.py run --root . --only testspec     # ②だけ全件
python <LR>/scripts/pipeline.py priority F-0012 --root .        # F-0012 に割り込ませる
```

AI の成果物は、あなたに届く前に機械の検査を通ります。

Table 9

検査	落とすもの
① の機械レビュー	実在しない行番号の引用・根拠なしの ● ・書きかけ・必須節の欠落・例外の検討漏れ
② の機械レビュー	● 仕様項目のケース漏れ・存在しない仕様 ID の参照・△未確定の残り
③ の突合	テストケース ID とテスト関数の過不足
④ のスタブ検知	空実装・NotImplementedError・TODO
⑤ 実行前の検証	上流改訂の伝搬漏れ (stale) ・freeze 後のテスト改変

7. 操作の入口と画面

画面（HTML サイト）は見るだけです。実行・承認のボタンはありません。返答のチャンネルは3つで、どれで返しても同じ結果になります。

Table 10

チャンネル	やり方
チャット	「F-0012 OK」「F-0012 修正: 〜」「ISSUE-004 は Yes」
ファイル記入	ISSUE の「回答（人が記入）」欄・docs/review-feedback.md に書く（次回起動時に AI が拾う）
CLI	review_actions.py approve / request-changes / adjudicate、variables.py approve、hazards.py add-policy、ledger unblock

```
python <LR>/scripts/render_site.py --root .      # docs/ → HTML（工程の区切りで実行）
python <LR>/scripts/serve_site.py --root .      # 閲覧（127.0.0.1 のみ）
python <LR>/scripts/build_viewer.py --root .    # レビューアへ配る単体 EXE（相手に環境不要）
```

pricing-batch リバースエンジニアリング WBS ドメイン知識 規約 ⑥完了検証 ⑦分析 新コード詳細(API) 🔍

pricing-batch リバースエンジニアリング WBS

公開
2026年7月25日

進捗サマリ

①spec	②test-spec	③test-code	④impl	⑤test
3/3	3/3	3/3	3/3	3/3

Open ISSUES（要人間判断）

なし 🚫

関数一覧（推奨着手順）

#	関数	依存	①spec	②test-spec	③test-code	④impl	⑤test
1	get_rate	なし	✓	✓	✓	✓	✓
2	round_val	なし	✓	✓	✓	✓	✓
3	calc_price	F-0001, F-0002	✓	✓	✓	✓	✓

⑦ 改善イタレーション

ID	分類	目的	対象	状態	達成
REF-001	refactor	get_rate のフォールバック読みを分離し複雑度を下げる	F-0001	verified	✓

目次

- 進捗サマリ
- Open ISSUES（要人間判断）
- 関数一覧（推奨着手順）
- ⑦ 改善イタレーション

図 1: WBS（進捗のホーム画面）。進捗サマリ・あなたへの質問（Open ISSUES）・関数一覧が 1 画面に集約され、各リンクから成果物へ移動できる

docs/templates/ と docs/prompts/ はサイトに載りません（成果物ではなく設定のためです）。

8. セットアップ

1. Claude Code CLI ・ Python 3.10+ ・ git ・ Quarto を用意する
2. skill を配置する（2 行とも必要。2 行目は `/legacy-1-spec` を認識させるため）

```
cp -r legacy-reverse <project>/./claude/skills/  
cp -r legacy-reverse/skills/* <project>/./claude/skills/
```

3. `hooks/settings-example.json` を `<project>/./claude/settings.json` にマージする（安全装置）
4. `ledger init-templates` で人が書くファイル一式の雛形を置く（④の中でも実行されます）
5. `/legacy-reverse` で状態を確認し、`/legacy-0-analyze` から始める

MCP サーバ（`mcp-servers/legacy-reverse-mcp`）の登録は任意です。登録すると スクリプトが型付きツールと呼ばれ、許可プロンプトとシェルの事故が減ります。

9. 困ったとき

パイプラインの仕組みを知っていれば読み解ける症状だけを挙げます（多くは「正常動作」です）。

Table 11

症状	意味	対処
❌ ISSUE-xxx が付いた	④⑤ループが上限に達し、裁定待ちで停止	ISSUE に答える → <code>ledger unblock F-xxxx</code> → ⑤を再トリガ
stale△が出た	上流（①）が改訂され、下流（②）が古い	②を再生成して再承認。ハッシュ連鎖の正常動作
△改変が出た	freeze 後にテストコードが変わった	意図した変更なら③で再freeze。心当たりが無ければ調査
tests/ の編集が拒否された	hook が正常動作している（④⑤中はテストを守る）	テスト側の疑義は ISSUE → 人の承認 → ②③の再生成が正規経路
機械レビュー NG と言われて承認できない	ハルシネーション・省略を検知（正常動作）	NG の理由を読み、AI に自己修正させてから承認する。承認はどの入口でも同じく拒否されます
無人バッチで、応答はあるのにファイルが更新されない	headless がツールの実行許可を得られていない	<code>settings.json</code> の <code>permissions.allow</code> を広げるか、閉じた環境なら <code>--skip-permissions</code> を付ける
バッチが「データ異常のため停止」	<code>data/*.json</code> が壊れている	<code>git diff data/functions.json</code> で直前の変更を確認して修復。hook (<code>guard_json.py</code>) を入れておくと、壊れる前に AI へ差し戻されます
連続実行が「連続 3 件失敗」で止まった	環境異常の疑いで安全停止	ログで API キー・レート・claude の解決を確認してから再開
図がソースのまま表示される	<code>quarto render docs</code> を直接叩いた	<code>render_site.py</code> で出し直す（Mermaid は影コピー経由でのみ描画されます）
サイト内検索に新しいページが出ない	差分レンダは検索索引を更新しない	節目で <code>python <LR>/scripts/render_site.py --root . --full</code> を 1 回

種別ごとの合本 PDF が要るときは `pdf_book.py` を使います（`quarto-typst-pdf skill` を `<LR>` の隣に置いてください。HTML だけなら不要です）。

```
python <LR>/scripts/pdf_book.py specs --root . --output pdf/関数仕様書.pdf --title 関数仕様書
```

付録: 用語

Table 12

語	意味
func-id (F-xxxx)	関数の通し番号。全成果物がこれで紐づく
SPEC-xxxx-NN	仕様書の機能詳細 1 項目の見出し ID。②のケースがこれを参照する
Confidence ●●●●	確認済み / 推測 / 仮定。●●●●にはレガシー行番号の根拠が必須
stale⚠	上流が改訂されて下流が古くなった状態。再生成が要る
🚫 blocked	人の裁定待ちで停止中。ISSUE に答えて <code>ledger unlock</code> で再開
EP-xxx	例外ポリシーの登録番号 (0 割などの扱いの決定)
V-xxxx	変数辞書の語義 ID。仕様書の IO 表に <code>[V-xxxx]</code> として転記される
dict-gate	語義が未承認の関数の①を止める仕組み